

ENGSE206 · การกำหนดความต้องการและการออกแบบทางซอฟต์แวร์

วิศวกรรมซอฟต์แวร์ · มหาวิทยาลัยเทคโนโลยีราชมงคลล้านนา (ดอยสะเก็ด)

User Story

ทฤษฎี หลักการ และการแปลงจากสถานการณ์จริง

เอกสารประกอบการสอนเชิงลึก — เจาะเฉพาะเรื่อง User Story



ทีมร่างสถานการณ์และ User Story ร่วมกัน

คำถามหลักของเอกสารนี้

เมื่อเจอสถานการณ์หรือความต้องการของผู้ใช้ในชีวิตจริง เราจะ "แปลง" มันให้กลายเป็น User Story ที่เขียนดี ตรวจสอบได้ และนำไปทำงานต่อได้อย่างไร?

1 User Story คืออะไร

User Story เป็นเทคนิคจากแนวคิด Agile / Extreme Programming (XP) ที่ใช้ "จับคุณค่า" ที่ผู้ใช้ต้องการ ในรูปประโยคสั้น ๆ แทนที่จะเขียนสเปกละเอียดทุกฟิลด์ตั้งแต่ต้น

แนวคิดดั้งเดิม — 3C's (Ron Jeffries)

C	ความหมาย
Card	เขียนสั้นพอลงบัตร/การ์ดใบเดียว — เป็นตัวแทนความจำ ไม่ใช่สเปกทั้งหมด
Conversation	รายละเอียดที่แท้จริงเกิดจาก "การพูดคุย" ระหว่างทีมกับผู้ใช้/เจ้าของผลิตภัณฑ์ ไม่ใช่จากตัวหนังสือ
Confirmation	ต้องมีวิธี "ยืนยัน" ว่าทำสำเร็จหรือยัง (ภายหลังจะกลายเป็น Acceptance Criteria)

ทำไมต้องใช้ User Story แทนการเขียนสเปกยาว ๆ แทนที่?

- ทำให้ทีมคุยกันเรื่อง "คุณค่าที่ผู้ใช้ต้องการ" ก่อน แทนที่จะรีบเลือกรายละเอียดที่ยังไม่แน่นอน
- ปรับเปลี่ยนได้ง่ายเมื่อความเข้าใจเปลี่ยนไประหว่างพัฒนา (Negotiable)
- เป็นหน่วยงานที่เล็กพอจะวางแผน ประมาณเวลา และทดสอบได้จริง

ในบริบทวิชานี้ User Story คือสิ่งที่แปลงมาจาก Requirement ที่ Ready ในขั้น Define/Specify (Week 05–08) ก่อนจะขยายต่อเป็น Use Case (บอกพฤติกรรมเป็นขั้นตอน) และ Acceptance Criteria (ทำให้ตรวจสอบได้)

2 รูปแบบมาตรฐาน (Anatomy)

รูปแบบที่ใช้กันแพร่หลายที่สุด (Connextra format):

ในฐานะ [role] ฉันต้องการ [goal] เพื่อ [value]



องค์ประกอบ 3 ส่วนของ User Story: Role, Goal, Value

ส่วน	ตอบคำถาม	กฎสำคัญ
Role (ใคร)	ใครเป็นผู้ได้ประโยชน์	ต้องเป็น actor ที่มีอยู่จริงในต้นทาง ห้ามประดิษฐ์ผู้ใช้ใหม่
Goal (อะไร)	ต้องการทำอะไร	ต้องเป็น "คุณค่า" ไม่ใช่ปุ่มหรือหน้าจอ (ไม่ผูก UI)
Value (เพื่ออะไร)	เหตุผลที่ goal มีความหมาย	ต้องตอบว่า "ทำไม" ให้ชัด ไม่ใช่แค่ทวนคำว่า goal

ตัวอย่าง

ในฐานะ Student Requester ฉันต้องการ ค้นหาทรัพยากรที่ว่าง ตามวัน/เวลา เพื่อ เลือกตัวเลือกก่อนส่งคำขอ

3 หลักการเขียนที่ดี — 3C's และ INVEST

หลัง 3C's แล้ว มาตรฐานที่ใช้ตรวจสอบ "คุณภาพ" ของ User Story แต่ละใบ คือหลัก INVEST (Bill Wake):



INVEST criteria: Independent, Negotiable, Valuable, Estimable, Small, Testable

อักษร	หลักการ	คำถามที่ใช้เช็ค
I — Independent	ทำ/ประเมินแยกจาก story อื่นได้	ถ้าตัด story นี้ออก ยังเข้าใจและทำได้ไหม?
N — Negotiable	รายละเอียดคุยปรับได้	มีค่าที่ผู้กดีไซน์ตายตัวไหม (สี, ตำแหน่งปุ่ม)?
V — Valuable	มีคุณค่าต่อ role ชัดเจน	ถ้าตัดออก role จะเสียประโยชน์อะไรที่จับต้องได้ไหม?
E — Estimable	ทีมประมาณขนาด/เวลาได้	ทีมพอเดางานคร่าว ๆ ได้ไหม หรือกำกวมเกินไป?
S — Small	เล็กพอทำเสร็จในสปรินต์/แล็บเดียว	ทำเสร็จภายในคาบ/รอบงานที่กำหนดได้จริงไหม?
T — Testable	ตรวจสอบด้วย Acceptance Criteria ได้	มีวิธี "เห็นผล" ว่าผ่านหรือไม่ผ่านหรือยัง?

ใช้งานจริง

ไม่ต้องท่องจำเป๊ะทุกตัว แต่ให้ใช้เป็น checklist ย้อนกลับ หลังเขียน story เสร็จ — ถ้าข้อไหนไม่ผ่าน ให้ย้อนไปแก้ role/goal/value ก่อนส่งต่อไปเขียน Use Case

4 User Story ต่างจาก Use Case และ Requirement อย่างไร

ความเข้าใจผิดที่พบบ่อยที่สุดคือคิดว่า User Story เป็น “Use Case แบบย่อ”

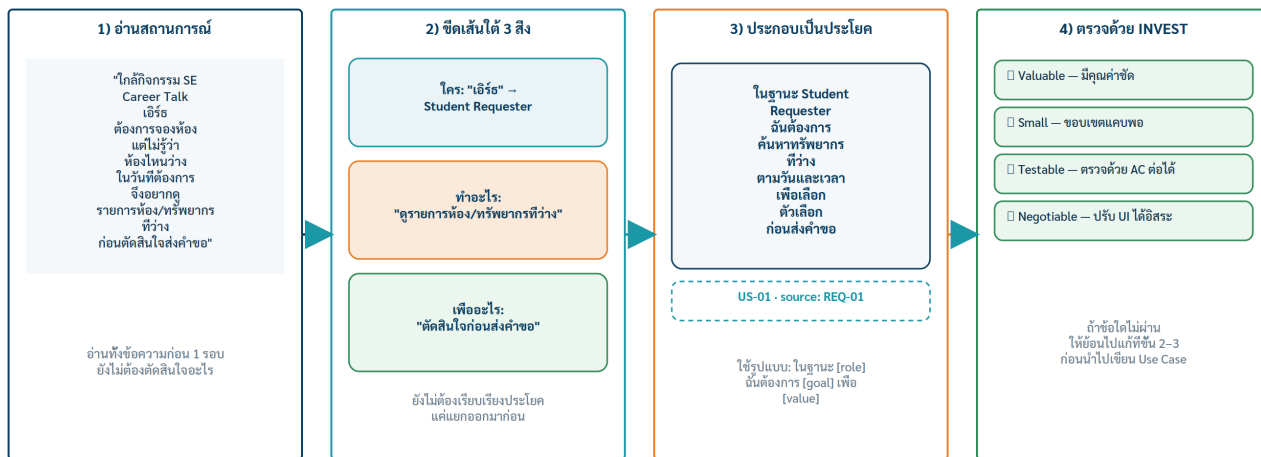
ทั้งที่จริงแล้วเป็นคนละเครื่องมือที่ตอบคนละคำถาม:

เครื่องมือ	ตอบคำถาม	รายละเอียด	ใช้ตอนไหน
Requirement	ระบบต้องทำอะไร (ภาพกว้าง)	ข้อความ + priority + status	Week 04–05 (Backlog)
User Story	ใคร/อะไร/ทำไม (คุณค่า)	สั้น เป็นจุดเริ่มบทสนทนา	Week 06 (จุดเริ่ม Define/Specify)
Use Case	ระบบกับ actor โต้ตอบกันอย่างไร	main flow, alternate flow	Week 06 (ขยายจาก Story ที่เลือก)
Acceptance Criteria	ทำถูกหรือยัง	Given/When/Then	Week 06 (ปิดท้ายให้ story ตรวจได้)

ในสัปดาห์นี้ ลำดับการทำงานคือ Requirement (Ready) → User Story → Use Case → Acceptance Criteria
— แต่ละขั้นไม่ได้แทนที่กัน เพียงแต่เพิ่มรายละเอียดคนละมิติ

5 ตัวอย่างจริง: จากสถานการณ์สู่ User Story

มาดูวิธี “แปลง” สถานการณ์จริงให้กลายเป็น User Story แบบทีละขั้น โดยใช้กรณีศึกษาของวิชานี้ — Campus Resource Booking (ระบบจองทรัพยากรในมหาวิทยาลัย)



กระบวนการ 4 ขั้น: อ่านสถานการณ์ → ขีดเส้นใต้ 3 สิ่ง → ประกอบเป็นประโยค → ตรวจด้วย INVEST

ขั้นที่ 1 — อ่านสถานการณ์

“ใกล้กิจกรรม SE Career Talk เอิร์ธต้องการจองห้องประชุมสำหรับจัดกิจกรรมนี้ แต่ไม่รู้ว่าจะจองที่ไหนว่างในวันที่ต้องการ จึงอยากเข้าไปดูรายการห้อง/ทรัพยากรที่ว่างในระบบก่อนตัดสินใจส่งคำขอ”

อ่านทั้งข้อความก่อน 1 รอบ ยังไม่ต้องรีบตัดสินใจอะไร

ขั้นที่ 2 — ขีดเส้นใต้ 3 สิ่ง (ใคร / ทำอะไร / เพื่ออะไร)

สิ่งที่ต้องหา	ข้อความในสถานการณ์	แปลงเป็น
ใคร (role)	“เอิร์ธ” (นักศึกษาที่ต้องการจองห้อง)	Student Requester
ทำอะไร (goal)	“ดูรายการห้อง/ทรัพยากรที่ว่าง”	ค้นหาทรัพยากรที่ว่างตามวัน/เวลา
เพื่ออะไร (value)	“ก่อนตัดสินใจส่งคำขอ”	เลือกตัวเลือกก่อนส่งคำขอ

ขั้นที่ 3 — ประกอบเป็นประโยคตามรูปแบบมาตรฐาน

US-01

ในฐานะ Student Requester ฉันต้องการ ค้นหาทรัพยากรที่ว่างตามวันและเวลา เพื่อเลือกตัวเลือกก่อนส่งคำขอ

source: REQ-01

■ ขั้นที่ 4 — ตรวจสอบด้วย INVEST

เกณฑ์	ผ่านไหม	เหตุผล
Valuable	✓	มีคุณค่าชัดเจนต่อ Student Requester
Small	✓	ขอบเขตแคบพอ (แค่ค้นหา ไม่รวมการส่งคำขอ)
Testable	✓	ตรวจสอบด้วย Acceptance Criteria ต่อได้
Negotiable	✓	ไม่ผูก UI เฉพาะเจาะจง ปรับได้อิสระ

ถ้าข้อใดไม่ผ่าน ให้ย้อนกลับไปแก้ที่ขั้น 2-3 ก่อนนำ story ไปเขียน Use Case ต่อ

■ เรื่องเดียวกัน แต่มองอีกมุม — เกิด Story ที่สองได้อย่างไร

สถานการณ์เดียวกันนี้ ถ้าขยายต่อว่า “หลังจากเอิร์ธเห็นห้องที่ว่างแล้ว เอิร์ธต้องกรอกวันเวลา ชื่อทรัพยากร และวัตถุประสงค์ เพื่อส่งคำขอเข้าสู่การพิจารณา” จะดึงออกมาเป็น story ที่สองได้ทันที:

US-02

ในฐานะ Student Requester ฉันต้องการส่งข้อมูลวัน เวลา ทรัพยากร และวัตถุประสงค์ เพื่อให้คำขอเข้าสู่การพิจารณา

source: REQ-02

ข้อสังเกตสำคัญ

สถานการณ์เดียวสามารถให้ story ได้มากกว่า 1 ใบ ถ้ามี “จุดตัดสินใจ” หรือ “เป้าหมายย่อย” มากกว่าหนึ่งจุด — สัญญาณคือ เมื่อ goal เปลี่ยน (ค้นหา → ส่งคำขอ) มักหมายถึง story ใหม่

6 ร่างแรก vs ฉบับแก้

การเขียน User Story ที่ดีมักไม่ได้มาจากร่างแรกเสมอไป มาดูตัวอย่างการปรับปรุงจริง:



เปรียบเทียบร่างแรกที่อ่อนกับฉบับแก้ที่แข็งแรง

จุดสังเกต	ร่างแรก (อ่อน)	ฉบับแก้ (แข็งแรง)
ข้อความ	ในฐานะผู้ใช้ ฉันต้องการปุ่มค้นหา เพื่อให้หน้าต่างสวยงาม	ในฐานะ Student Requester ฉันต้องการค้นหาทรัพยากรที่ว่างตามวัน/เวลา เพื่อเลือกก่อนส่งคำขอ
Role	X ลอย ไม่ใช่ actor จริง	✓ มาจาก backlog จริง (Student Requester)
Goal	X เป็น UI (“ปุ่มค้นหา”) ไม่ใช่คุณค่า	✓ เป็นคุณค่า ไม่ใช่ UI
Value	X ไม่เชื่อมกับผู้ใช้จริง	✓ ตอบว่า “ทำไม” ชัดเจน
Trace	X trace ไป REQ ไม่ได้	✓ trace ถึง REQ-01 ได้

บทเรียน

ทุกครั้งที่เห็นคำว่า “ปุ่ม” “สี” “หน้าจอ” “หน้าต่างสวยงาม” ใน goal/value ให้สงสัยว่า story นั้นกำลังบรรยาย UI ไม่ใช่คุณค่า — ให้ถามซ้ำว่า “ทำไม” แล้วผู้ใช้ได้ประโยชน์อะไรจริง ๆ

7 ความเข้าใจผิดที่พบบ่อย

ความเข้าใจผิด	วิธีคิดที่ถูกต้อง
User Story = Use Case แบบย่อ	คนละคำถาม — story ตอบ ใคร/อะไร/ทำไม ส่วน use case อธิบายปฏิสัมพันธ์เป็นขั้นตอน
ยิ่งเขียนละเอียดยิ่งดี	Story ควรสั้น (Card) — รายละเอียดเกิดจาก "การพูดคุย" ไม่ใช่จากตัวหนังสือที่ยาวขึ้น
Role คือใครก็ได้ที่ "ใช้ระบบ"	Role ต้องเป็น actor ที่มีอยู่จริงในต้นทาง ไม่ใช่ผู้ใช้สมมติทั่วไป
Value คือการทวนคำว่า goal	Value ต้องตอบ "ทำไม" อีกชั้นหนึ่ง ไม่ใช่แค่ทวน goal
1 สถานการณ์ = 1 story เสมอ	สถานการณ์เดียวอาจมีหลาย goal ซ่อนอยู่ ทำให้ได้หลาย story
Story ที่ดีไม่ต้องแก้	ร่างแรกมักต้องปรับ — ใช้ INVEST เป็นเครื่องมือย้อนกลับไปแก้ก่อนส่งต่อ

8 แบบฝึกหัดท้ายบท

ลองฝึกแปลงสถานการณ์ต่อไปนี้เป็น User Story ด้วยตนเอง โดยทำตาม 4 ขั้นตอนในหัวข้อ 5:

"ทีมจัดกิจกรรม SE Career Talk จองห้องไว้แล้ว แต่ใกล้วันงานต้องการทราบว่าค่าของที่ส่งไปได้รับการอนุมัติหรือยัง เพราะถ้ายังไม่อนุมัติจะต้องหาห้องสำรอง"

คำใบ้

ลองขีดเส้นใต้ 3 สิ่งก่อน (ใคร / ทำอะไร / เพื่ออะไร) แล้วประกอบเป็นประโยคตามรูปแบบมาตรฐาน จากนั้นตรวจด้วย INVEST — สังเกตว่า goal ของสถานการณ์นี้ต่างจาก US-01/US-02 อย่างไร (คำใบ้: เกี่ยวกับ REQ-03 การติดตามสถานะ)

9 สรุป

- User Story จับ **คุณค่า** ของผู้ใช้ในรูปแบบสั้น ๆ (Card) เพื่อเปิดบทสนทนา (Conversation) และมีวิธียืนยันผล (Confirmation)
- ใช้รูปแบบ **ในฐานะ [role] ฉันต้องการ [goal] เพื่อ [value]**
- ตรวจสอบภาพด้วย **INVEST** ก่อนส่งต่อไปเขียน Use Case
- การแปลงจากสถานการณ์จริงทำได้เป็น 4 ขั้นตอน: อ่าน → ชีตเส้นใต้ 3 สิ่ง → ประกอบประโยค → ตรวจสอบ INVEST
- Story ที่ดีมักผ่านการแก้ไขจากร่างแรก — ไม่ใช่เรื่องผิดปกติ
- ทุก Story ต้อง **trace** กลับไปยัง Requirement ที่ Ready ได้เสมอ

เอกสารนี้เป็นส่วนหนึ่งของชุดสื่อการสอน ENGSE206 Week 06 — ใช้ประกอบคู่กับเอกสาร "Use Case Diagram — ทฤษฎี หลักการ และการแปลงจากสถานการณ์จริง"